Algorithmic Challenges: Suffix Array

Michael Levin

Department of Computer Science and Engineering
University of California, San Diego

String Processing and Pattern Matching Algorithms

Algorithms and Data Structures

Outline

1. Suffix Array
2. General Construction Strategy
3. Initialization
4. Sort Doubled Cyclic Shifts
5. Implement SortDoubled
6. Updating Classes
7. Implement UpdateClasses
8. Building Suffix Array

Construct Suffix Array

Input: String S

Output: All suffixes of S in lexicographic order

Alphabet

We assume the alphabet is ordered, that is, for any two different characters in the alphabet one of them is considered smaller than another. For example, in English

$$\text{'a'} < \text{'b'} < \text{'c'} < \cdots < \text{'z'}$$

Definition

String S is lexicographically smaller than string T if $S \neq T$ and there exist such i that:

- $0 \leq i \leq |S|$
- $S[0..i-1] = T[0..i-1]$
  (assume $S[0..-1]$ is an empty string)
- Either $i = |S|$ (then S is a prefix of T)
  or $S[i] < T[i]$

Examples

“ab” $<$ “bc” $(i = 0)$

“abc” $<$ “abd” $(i = 2)$

“abc” $<$ “abcd” $(i = 3)$

Suffix Array Example

$S = ababaa$

All suffixes:

ababaa

babaa

abaa

baa

aa

a

Suffix Array Example

$S = ababaa$

Suffixes in lexicographic order:

a

aa

abaa

ababaa

baa

babaa

Avoiding Prefix Rule

- Inconvenient rule: if S is a prefix of T, then $S < T$
- Append special character '\$' smaller than all other characters to the end of all strings
- If S is a prefix of T, then $S\$$ differs from $T\$$ in position $i = |S|$, and $\$ < T[|S|]$, so $S\$ < T\$$

Example

$S =$ "ababaa" $\Rightarrow S' =$ "ababaa\$"

Suffixes in lexicographic order:

\$

a\$

aa\$

abaa\$

ababaa\$

baa\$

babaa\$

Example

$S =$ “ababaa” $\Rightarrow S' =$ “ababaa$”

Suffixes in lexicographic order:

a

aa

abaa

ababaa

baa

babaa

Storing Suffix Array

- Total length of all suffixes is $1 + 2 + \cdots + |S| = \Theta(|S|^2)$

Storing Suffix Array

- Total length of all suffixes is
  $1 + 2 + \cdots + |S| = \Theta(|S|^2)$
- Storing them all is too much memory

Storing Suffix Array

- Total length of all suffixes is $1 + 2 + \cdots + |S| = \Theta(|S|^2)$
- Storing them all is too much memory
- Store the order of suffixes $O(|S|)$

Storing Suffix Array

- Total length of all suffixes is $1 + 2 + \cdots + |S| = \Theta(|S|^2)$
- Storing them all is too much memory
- Store the order of suffixes $O(|S|)$
- Suffix array is this order

Example

$S = ababaa\$$

Example

$S = ababaa\$$

Suffixes are numbered by their starting positions: $ababaa\$$ is 0, $abaa\$$ is 2

Example

$S = ababaa\$$

Suffixes are numbered by their starting positions: $ababaa\$$ is 0, $abaa\$$ is 2

Suffix array: $order = []$

Example

$S = ababaa\$$

Suffixes are numbered by their starting positions: *ababaa*\$ is 0, *abaa*\$ is 2

Suffix array: $order = [6]$

Example

$S = ababaa\$$

Suffixes are numbered by their starting positions: *ababaa*\$ is 0, *abaa*\$ is 2

Suffix array: $order = [6, 5]$

Example

$S = ababaa\$$

Suffixes are numbered by their starting positions: *ababaa*\$ is 0, *abaa*\$ is 2

Suffix array: $order = [6, 5, 4]$

Example

$S = ababaa\$$

Suffixes are numbered by their starting positions: *ababaa*\$ is 0, *abaa*\$ is 2

Suffix array: $order = [6, 5, 4, 2]$

Example

$S = ababaa\$$

Suffixes are numbered by their starting positions: *ababaa*\$ is 0, *abaa*\$ is 2

Suffix array: $order = [6, 5, 4, 2, 0]$

Example

$S = ababaa\$$

Suffixes are numbered by their starting positions: $ababaa\$$ is 0, $abaa\$$ is 2

Suffix array: $order = [6, 5, 4, 2, 0, 3]$

Example

$S = ababaa\$$

Suffixes are numbered by their starting positions: *ababaa*\$ is 0, *abaa*\$ is 2

Suffix array: $order = [6, 5, 4, 2, 0, 3, 1]$

Example

$S = ababaa\$$

Suffixes are numbered by their starting positions: $ababaa\$$ is 0, $abaa\$$ is 2

Suffix array: $order = [6, 5, 4, 2, 0, 3, 1]$

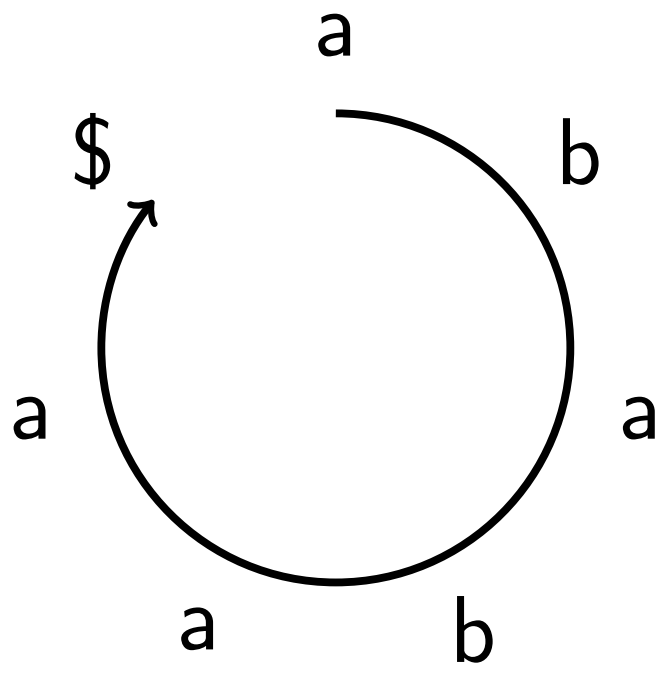
- OK, you know how to store suffix array

- OK, you know how to store suffix array
- But how to construct it?

Outline

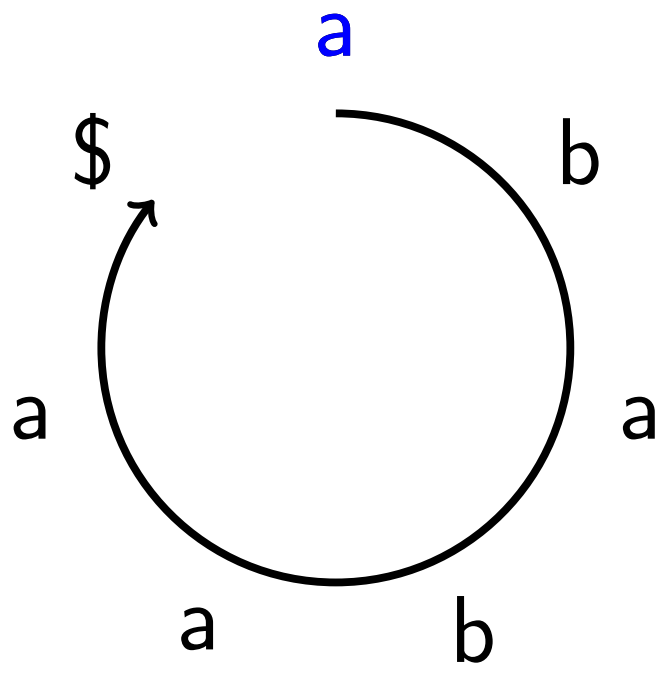
1. Suffix Array
2. General Construction Strategy
3. Initialization
4. Sort Doubled Cyclic Shifts
5. Implement SortDoubled
6. Updating Classes
7. Implement UpdateClasses
8. Building Suffix Array

Sorting Cyclic Shifts



Sorting Cyclic Shifts

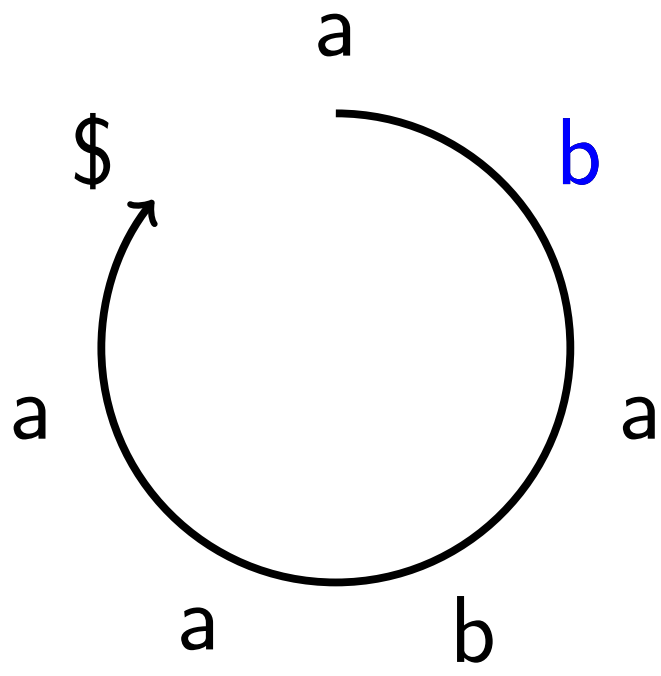
ababaa$



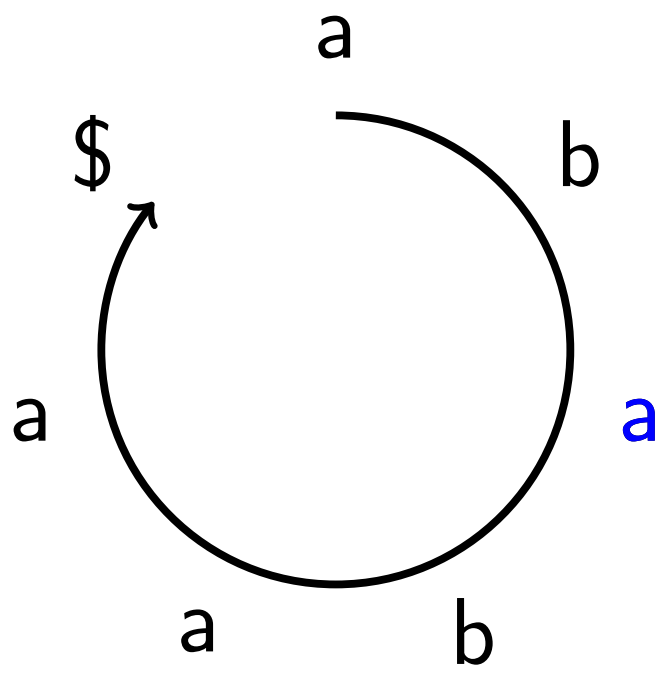
Sorting Cyclic Shifts

ababaa$

babaa$a



Sorting Cyclic Shifts

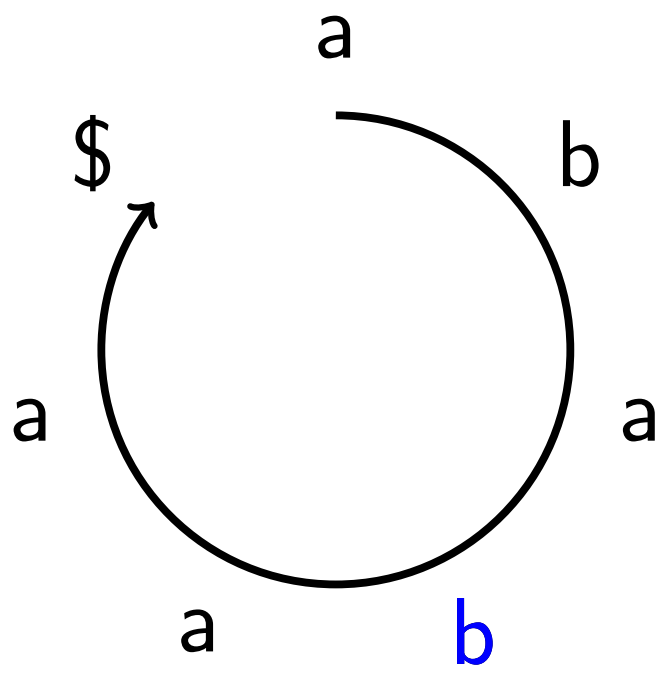


ababaa$

babaa$a

abaa$ab

Sorting Cyclic Shifts



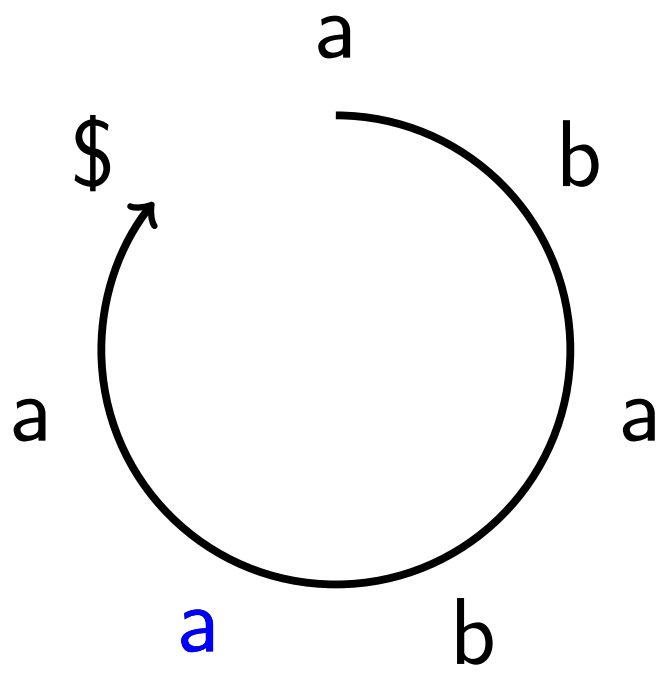
ababaa$

babaa$a

abaa$ab

baa$aba

Sorting Cyclic Shifts



ababaa$

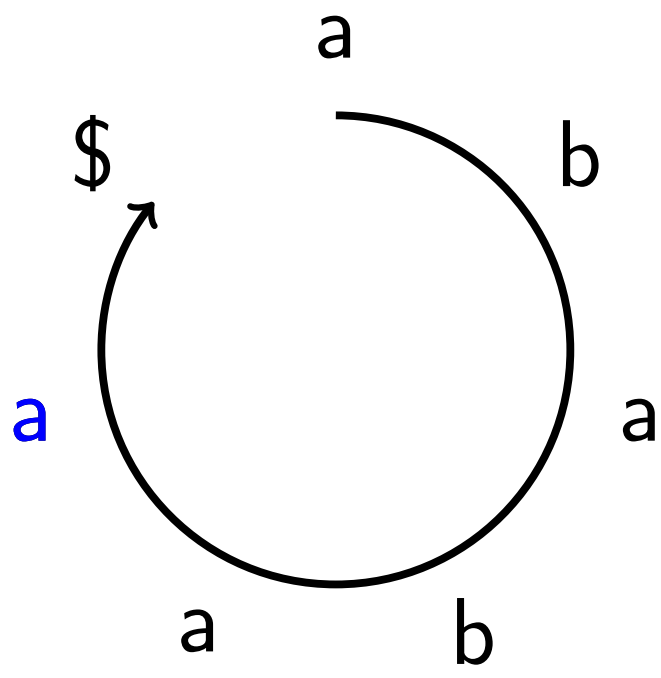
babaa$a

abaa$ab

baa$aba

aa$abab

Sorting Cyclic Shifts



ababaa$

babaa$a

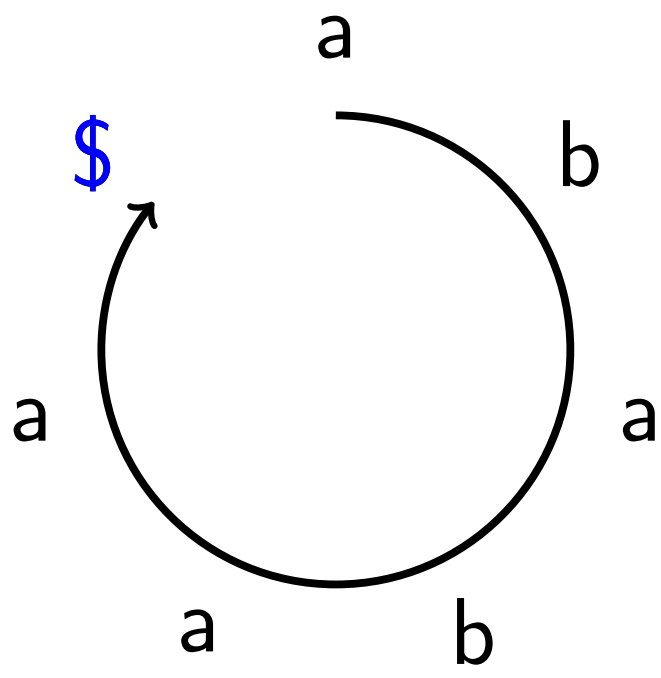
abaa$ab

baa$aba

aa$abab

a$ababa

Sorting Cyclic Shifts



ababaa$

babaa$a

abaa$ab

baa$aba

aa$abab

a$ababa

$ababaa

Sorting Cyclic Shifts

ababaa$

babaa$a

abaa$ab

baa$aba

aa$abab

a$ababa

$ababaa

Sorting Cyclic Shifts

ababaa$	$ababaa
babaa$a	a$ababa
abaa$ab	aa$abab
baa$aba	abaa$ab
aa$abab	ababaa$
a$ababa	baa$aba
$ababaa	babaa$a

Sorting Cyclic Shifts

ababaa$	$ababaa
babaa$a	a$ababa
abaa$ab	aa$abab
baa$aba	abaa$ab
aa$abab	ababaa$
a$ababa	baa$aba
$ababaa	babaa$a

Sorting Cyclic Shifts

ababaa$	$ababaa	$
babaa$a	a$ababa	a$
abaa$ab	aa$abab	aa$
baa$aba	abaa$ab	abaa$
aa$abab	ababaa$	ababaa$
a$ababa	baa$aba	baa$
$ababaa	babaa$a	babaa$

Lemma

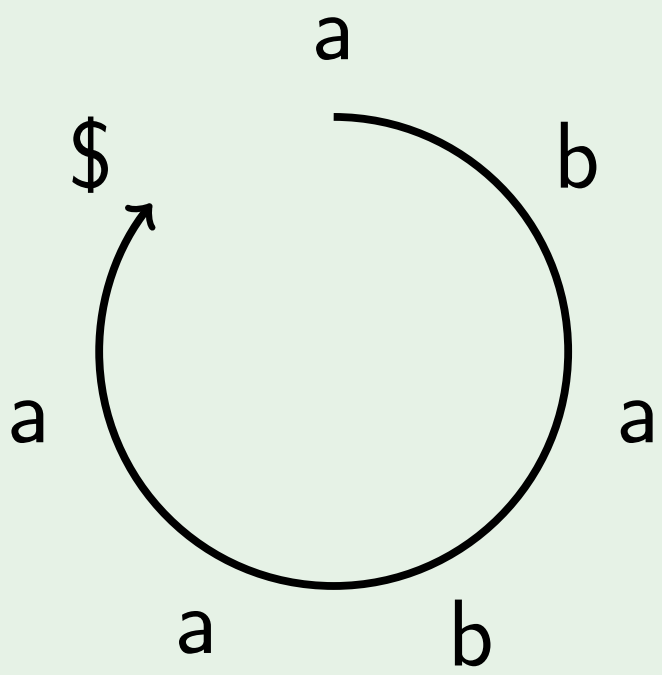
After adding to the end of string S character \$ which is smaller than all other characters, sorting cyclic shifts of S and suffixes of S is equivalent.

Partial Cyclic Shifts

Definition

Substrings of cyclic string S are called partial cyclic shifts of S

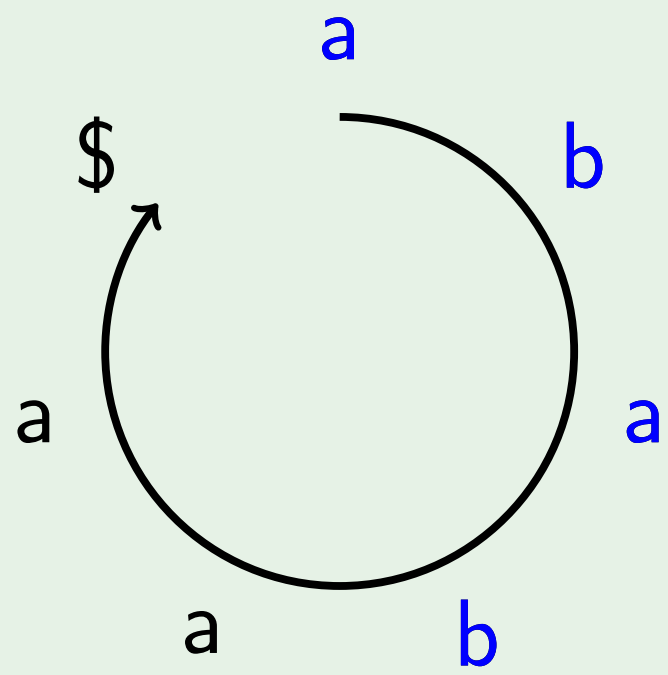
Partial Cyclic Shifts Example



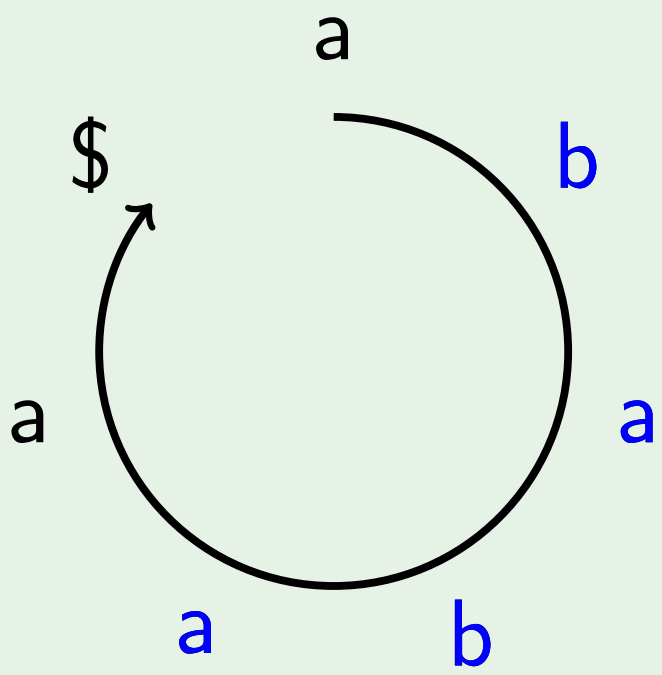
Partial Cyclic Shifts Example

Cyclic shifts of length 4:

abab



Partial Cyclic Shifts Example

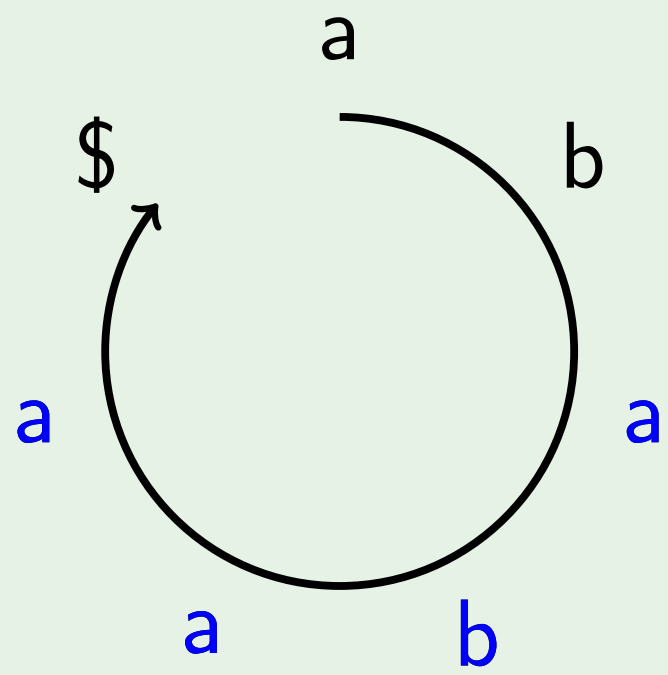


Cyclic shifts of length 4:

abab

baba

Partial Cyclic Shifts Example



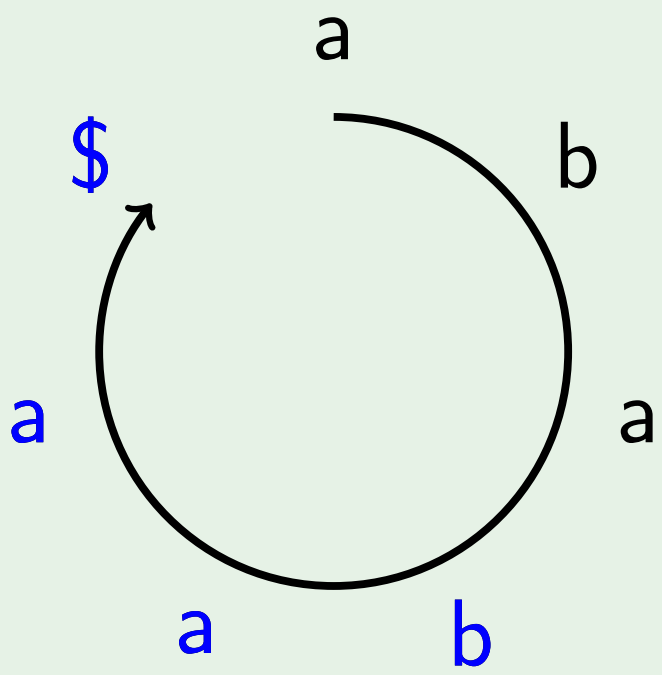
Cyclic shifts of length 4:

abab

baba

abaa

Partial Cyclic Shifts Example



Cyclic shifts of length 4:

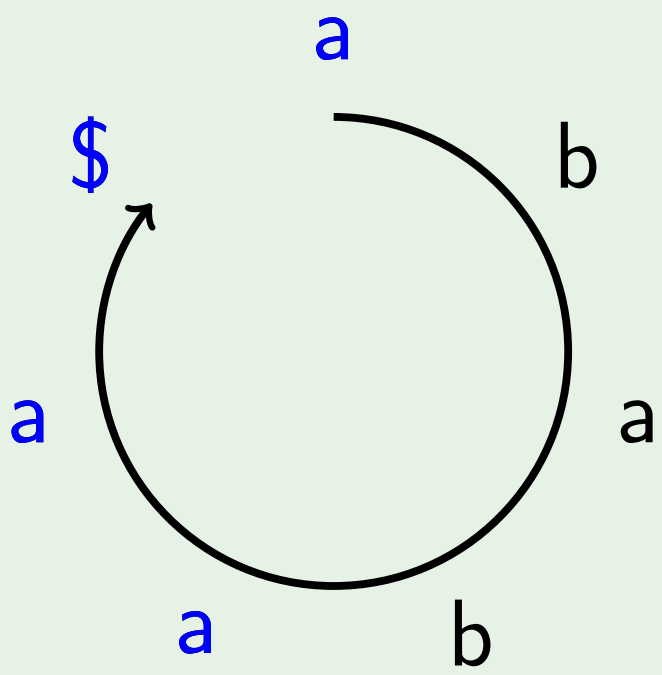
abab

baba

abaa

baa$

Partial Cyclic Shifts Example



Cyclic shifts of length 4:

abab

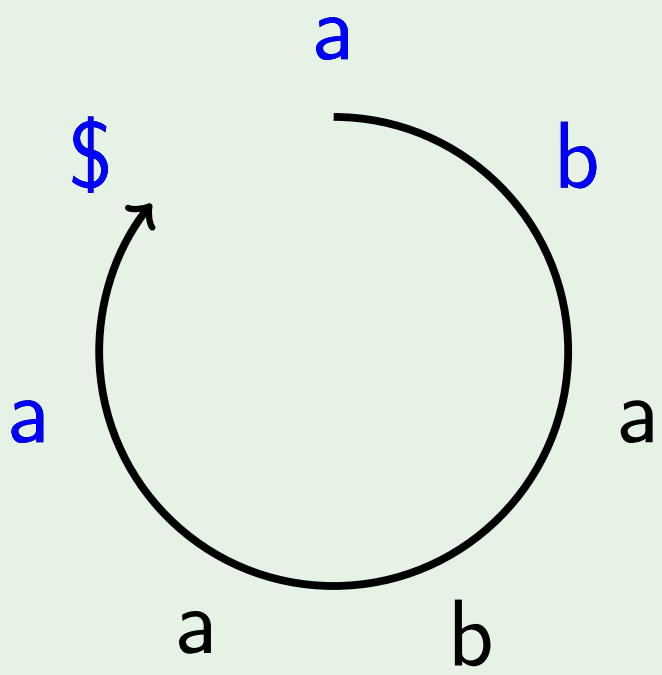
baba

abaa

baa$

aa$a

Partial Cyclic Shifts Example



Cyclic shifts of length 4:

abab

baba

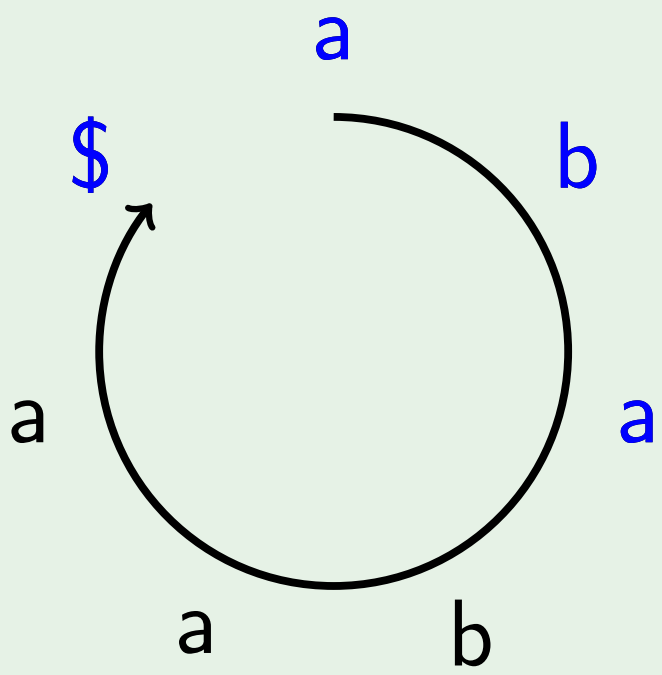
abaa

baa$

aa$a

a$ab

Partial Cyclic Shifts Example



Cyclic shifts of length 4:

abab

baba

abaa

baa$

aa$a

a$ab

$aba

General strategy

- Start with sorting single characters of S

General strategy

- Start with sorting single characters of S
- Cyclic shifts of length $L = 1$ sorted

General strategy

- Start with sorting single characters of S
- Cyclic shifts of length $L = 1$ sorted
- While $L < |S|$, sort shifts of length $2L$

General strategy

- Start with sorting single characters of S
- Cyclic shifts of length $L = 1$ sorted
- While $L < |S|$, sort shifts of length $2L$
- If $L \geq |S|$, cyclic shifts of length L sort the same way as cyclic shifts of length $|S|$

Example

$S = ababaa\$$

Example

$S = ababaa\$$

6	$
0	a
2	a
4	a
5	a
1	b
3	b

Example

$S = ababaa\$$

6	\$	$order = [6, 0, 2, 4, 5, 1, 3]$
0	a	
2	a	
4	a	
5	a	
1	b	
3	b	

Example

$S = ababaa\$$

6	$\$a$
5	$a\$$
4	aa
0	ab
2	ab
1	ba
3	ba

Example

$S = ababaa\$$

6	$\$a$	$order = [6, 5, 4, 0, 2, 1, 3]$
5	$a\$$	
4	aa	
0	ab	
2	ab	
1	ba	
3	ba	

Example

$S = ababaa\$$

6 $\$aba$

5 $a\$ab$

4 $aa\$a$

2 $abaa$

0 $abab$

3 $baa\$$

1 $baba$

Example

$S = ababaa\$$

6	$\$aba$	$order = [6, 5, 4, 2, 0, 3, 1]$
5	$a\$ab$	
4	$aa\$a$	
2	$abaa$	
0	$abab$	
3	$baa\$$	
1	$baba$	

Example

$S = ababaa\$$

6 $\$ababaa\$$

5 $a\$ababaa$

4 $aa\$ababa$

2 $abaa\$aba$

0 $ababaa\$a$

3 $baa\$abab$

1 $babaa\$ab$

Example

$S = ababaa\$$

6 $\$ababaa\$$ $order = [6, 5, 4, 2, 0, 3, 1]$

5 $a\$ababaa$

4 $aa\$ababa$

2 $abaa\$aba$

0 $ababaa\$a$

3 $baa\$abab$

1 $babaa\$ab$

Example

$S = ababaa\$$

6 $\$ababaa\$$ $order = [6, 5, 4, 2, 0, 3, 1]$

5 $a\$ababaa$

4 $aa\$ababa$

2 $abaa\$aba$

0 $ababaa\$a$

3 $baa\$abab$

1 $babaa\$ab$

Example

$S = ababaa\$$

6	$
5	$a\$$
4	$aa\$$
2	$abaa\$$
0	$ababaa\$$
3	$baa\$$
1	$babaa\$$

$order = [6, 5, 4, 2, 0, 3, 1]$

BuildSuffixArray(S)

```
order ← SortCharacters(S)
class ← ComputeCharClasses(S, order)
L ← 1
while L < |S|:
    order ← SortDoubled(S, L, order, class)
    class ← UpdateClasses(order, class, L)
    L ← 2L
return order
```

BuildSuffixArray(S)

```
order ← SortCharacters(S)
class ← ComputeCharClasses(S, order)
L ← 1
while L < |S|:
    order ← SortDoubled(S, L, order, class)
    class ← UpdateClasses(order, class, L)
    L ← 2L
return order
```

BuildSuffixArray(S)

```
order ← SortCharacters(S)
class ← ComputeCharClasses(S, order)
L ← 1
while L < |S|:
    order ← SortDoubled(S, L, order, class)
    class ← UpdateClasses(order, class, L)
    L ← 2L
return order
```

BuildSuffixArray(S)

```
order ← SortCharacters(S)
class ← ComputeCharClasses(S, order)
L ← 1
while L < |S|:
    order ← SortDoubled(S, L, order, class)
    class ← UpdateClasses(order, class, L)
    L ← 2L
return order
```

BuildSuffixArray(S)

```
order ← SortCharacters(S)
class ← ComputeCharClasses(S, order)
L ← 1
while L < |S|:
    order ← SortDoubled(S, L, order, class)
    class ← UpdateClasses(order, class, L)
    L ← 2L
return order
```

BuildSuffixArray(S)

```
order ← SortCharacters(S)
class ← ComputeCharClasses(S, order)
L ← 1
while L < |S|:
    order ← SortDoubled(S, L, order, class)
    class ← UpdateClasses(order, class, L)
    L ← 2L
return order
```

Outline

1. Suffix Array
2. General Construction Strategy
3. Initialization
4. Sort Doubled Cyclic Shifts
5. Implement SortDoubled
6. Updating Classes
7. Implement UpdateClasses
8. Building Suffix Array

Sorting single characters

- Alphabet Σ has $|\Sigma|$ different characters

Sorting single characters

- Alphabet Σ has $|\Sigma|$ different characters
- Use counting sort to compute *order* of characters

SortCharacters(S)

```
order ← array of size |S|
count ← zero array of size |Σ|
for i from 0 to |S| − 1:
    count[S[i]] ← count[S[i]] + 1
for j from 1 to |Σ| − 1:
    count[j] ← count[j] + count[j − 1]
for i from |S| − 1 down to 0:
    c ← S[i]
    count[c] ← count[c] − 1
    order[count[c]] ← i
return order
```

SortCharacters(S)

```
order ← array of size |S|
count ← zero array of size |Σ|
for i from 0 to |S| − 1:
   count[S[i]] ← count[S[i]] + 1
for j from 1 to |Σ| − 1:
   count[j] ← count[j] + count[j − 1]
for i from |S| − 1 down to 0:
   c ← S[i]
   count[c] ← count[c] − 1
   order[count[c]] ← i
return order
```

SortCharacters(S)

```
order ← array of size |S|
count ← zero array of size |Σ|
for i from 0 to |S| − 1:
    count[S[i]] ← count[S[i]] + 1
for j from 1 to |Σ| − 1:
    count[j] ← count[j] + count[j − 1]
for i from |S| − 1 down to 0:
    c ← S[i]
    count[c] ← count[c] − 1
    order[count[c]] ← i
return order
```

SortCharacters(S)

```
order ← array of size |S|
count ← zero array of size |Σ|
for i from 0 to |S| − 1:
    count[S[i]] ← count[S[i]] + 1
for j from 1 to |Σ| − 1:
    count[j] ← count[j] + count[j − 1]
for i from |S| − 1 down to 0:
    c ← S[i]
    count[c] ← count[c] − 1
    order[count[c]] ← i
return order
```

SortCharacters(S)

```
order ← array of size |S|
count ← zero array of size |Σ|
for i from 0 to |S| − 1:
    count[S[i]] ← count[S[i]] + 1
for j from 1 to |Σ| − 1:
    count[j] ← count[j] + count[j − 1]
for i from |S| − 1 down to 0:
    c ← S[i]
    count[c] ← count[c] − 1
    order[count[c]] ← i
return order
```

SortCharacters(S)

```
order ← array of size |S|
count ← zero array of size |Σ|
for i from 0 to |S| − 1:
    count[S[i]] ← count[S[i]] + 1
for j from 1 to |Σ| − 1:
    count[j] ← count[j] + count[j − 1]
for i from |S| − 1 down to 0:
    c ← S[i]
    count[c] ← count[c] − 1
    order[count[c]] ← i
return order
```

Lemma

Running time of `SortCharacters` is $O(|S| + |\Sigma|)$.

Proof

We know this is the running time of the counting sort for $|S|$ items that can take $|\Sigma|$ different values. □

Equivalence classes

- C_i — partial cyclic shift of length L starting in i
- C_i can be equal to C_j — then they are in one equivalence class
- Compute $class[i]$ — number of *different* cyclic shifts of length L that are strictly smaller than C_i
- $C_i == C_j \Leftrightarrow class[i] == class[j]$

Example

$S = ababaa\$$

6	\$	$order = [6, 0, 2, 4, 5, 1, 3]$
0	a	$class = [\ ,\ ,\ ,\ ,\ ,\ ,\ ]$
2	a	
4	a	
5	a	
1	b	
3	b	

Example

$S = ababaa\$$

6	$	$order = [6, 0, 2, 4, 5, 1, 3]$
0	a	$class = [\ ,\ ,\ ,\ ,\ ,\ ,\ ]$
2	a	
4	a	
5	a	
1	b	
3	b	

Example

$S = ababaa\$$

6	$	$order = [6, 0, 2, 4, 5, 1, 3]$
0	a	$class = [\ ,\ ,\ ,\ ,\ ,\ ,0]$
2	a	
4	a	
5	a	
1	b	
3	b	

Example

$S = ababaa\$$

6	\$	$order = [6, 0, 2, 4, 5, 1, 3]$
0	a	$class = [\ ,\ ,\ ,\ ,\ ,\ ,0]$
2	a	
4	a	
5	a	
1	b	
3	b	

Example

$S = ababaa\$$

6	$	$order = [6, 0, 2, 4, 5, 1, 3]$
0	a	$class = [1, \ , \ , \ , \ , \ , 0]$
2	a	
4	a	
5	a	
1	b	
3	b	

Example

$S = ababaa\$$

6	\$	$order = [6, 0, 2, 4, 5, 1, 3]$
0	a	$class = [1, \ , \ , \ , \ , \ , 0]$
2	a	
4	a	
5	a	
1	b	
3	b	

Example

$S = ababaa\$$

6	$	$order = [6, 0, 2, 4, 5, 1, 3]$
0	a	$class = [1, \ , 1, \ , \ , \ , 0]$
2	a	
4	a	
5	a	
1	b	
3	b	

Example

$S = ababaa\$$

6	$	$order = [6, 0, 2, 4, 5, 1, 3]$
0	a	$class = [1, \ , 1, \ , \ , \ , 0]$
2	a	
4	a	
5	a	
1	b	
3	b	

Example

$S = ababaa\$$

6	$	$order = [6, 0, 2, 4, 5, 1, 3]$
0	a	$class = [1, \ , 1, \ , 1, \ , 0]$
2	a	
4	a	
5	a	
1	b	
3	b	

Example

$S = ababaa\$$

6	$	$order = [6, 0, 2, 4, 5, 1, 3]$
0	a	$class = [1, \ , 1, \ , 1, \ , 0]$
2	a	
4	a	
5	a	
1	b	
3	b	

Example

$S = ababaa\$$

6	$	$order = [6, 0, 2, 4, 5, 1, 3]$
0	a	$class = [1, \ , 1, \ , 1, 1, 0]$
2	a	
4	a	
5	a	
1	b	
3	b	

Example

$S = ababaa\$$

6	$	$order = [6, 0, 2, 4, 5, 1, 3]$
0	a	$class = [1, \ , 1, \ , 1, 1, 0]$
2	a	
4	a	
5	a	
1	b	
3	b	

Example

$S = ababaa\$$

6	\$	$order = [6, 0, 2, 4, 5, 1, 3]$
0	a	$class = [1, 2, 1, \ , 1, 1, 0]$
2	a	
4	a	
5	a	
1	b	
3	b	

Example

$S = ababaa\$$

6	$	$order = [6, 0, 2, 4, 5, 1, 3]$
0	a	$class = [1, 2, 1, \ , 1, 1, 0]$
2	a	
4	a	
5	a	
1	b	
3	b	

Example

$S = ababaa\$$

6	$	$order = [6, 0, 2, 4, 5, 1, 3]$
0	a	$class = [1, 2, 1, 2, 1, 1, 0]$
2	a	
4	a	
5	a	
1	b	
3	b	

Example

$S = ababaa\$$

6	\$	$order = [6, 0, 2, 4, 5, 1, 3]$
0	a	$class = [1, 2, 1, 2, 1, 1, 0]$
2	a	
4	a	
5	a	
1	b	
3	b	

ComputeCharClasses(S, $order$)

$class \leftarrow$ array of size $|S|$
$class[order[0]] \leftarrow 0$
for i from 1 to $|S| - 1$:
  if $S[order[i]] \neq S[order[i-1]]$:
    $class[order[i]] = class[order[i-1]] + 1$
  else:
    $class[order[i]] = class[order[i-1]]$
return $class$

ComputeCharClasses(S, $order$)

```
class ← array of size |S|
class[order[0]] ← 0
for i from 1 to |S| − 1:
    if S[order[i]] ≠ S[order[i − 1]]:
        class[order[i]] = class[order[i − 1]] + 1
    else:
        class[order[i]] = class[order[i − 1]]
return class
```

ComputeCharClasses(S, $order$)

```
class ← array of size |S|
class[order[0]] ← 0
for i from 1 to |S| − 1:
    if S[order[i]] ≠ S[order[i − 1]]:
        class[order[i]] = class[order[i − 1]] + 1
    else:
        class[order[i]] = class[order[i − 1]]
return class
```

ComputeCharClasses(S, $order$)

```
class ← array of size |S|
class[order[0]] ← 0
for i from 1 to |S| − 1:
    if S[order[i]] ≠ S[order[i − 1]]:
        class[order[i]] = class[order[i − 1]] + 1
    else:
        class[order[i]] = class[order[i − 1]]
return class
```

ComputeCharClasses(S, *order*)

```
class ← array of size |S|
class[order[0]] ← 0
for i from 1 to |S| − 1:
    if S[order[i]] ≠ S[order[i − 1]]:
        class[order[i]] = class[order[i − 1]] + 1
    else:
        class[order[i]] = class[order[i − 1]]
return class
```

ComputeCharClasses(S, $order$)

```
class ← array of size |S|
class[order[0]] ← 0
for i from 1 to |S| − 1:
    if S[order[i]] ≠ S[order[i − 1]]:
        class[order[i]] = class[order[i − 1]] + 1
    else:
        class[order[i]] = class[order[i − 1]]
return class
```

ComputeCharClasses(S, $order$)

```
class ← array of size |S|
class[order[0]] ← 0
for i from 1 to |S| − 1:
    if S[order[i]] ≠ S[order[i − 1]]:
        class[order[i]] = class[order[i − 1]] + 1
    else:
        class[order[i]] = class[order[i − 1]]
return class
```

Lemma

The running time of `ComputeCharClasses` is $O(|S|)$.

Proof

One for loop with $O(|S|)$ iterations. □

Outline

1. Suffix Array
2. General Construction Strategy
3. Initialization
4. Sort Doubled Cyclic Shifts
5. Implement SortDoubled
6. Updating Classes
7. Implement UpdateClasses
8. Building Suffix Array

Idea

- C_i — cyclic shift of length L starting in i

Idea

- C_i — cyclic shift of length L starting in i
- C'_i — doubled cyclic shift starting in i

Idea

- C_i — cyclic shift of length L starting in i
- C'_i — doubled cyclic shift starting in i
- $C'_i = C_i C_{i+L}$ — concatenation of strings

Idea

- C_i — cyclic shift of length L starting in i
- C'_i — doubled cyclic shift starting in i
- $C'_i = C_i C_{i+L}$ — concatenation of strings
- To compare C'_i with C'_j, it's sufficient to compare C_i with C_j and C_{i+L} with C_{j+L}

Example

$S = ababaa\$$

$L = 2$

$i = 2$

$C_i = C_2 = ab$

$C_{i+L} = C_{2+2} = C_4 = aa$

$C'_i = C'_2 = abaa = C_2 C_4$

Sorting pairs

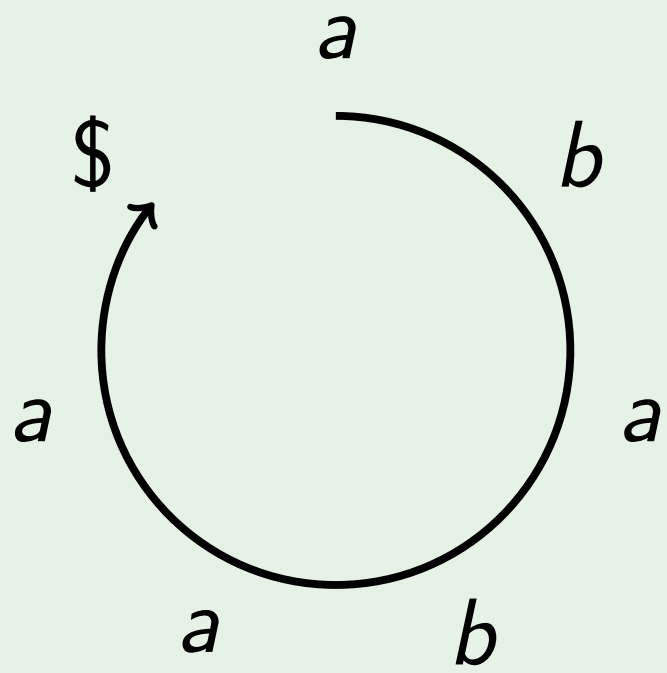
- First sort by second element of pair

Sorting pairs

- First sort by second element of pair
- Then **stable** sort by first element of pair

Example

$L = 2$



$C_6 = \$a$

$C_5 = a\$$

$C_4 = aa$

$C_0 = ab$

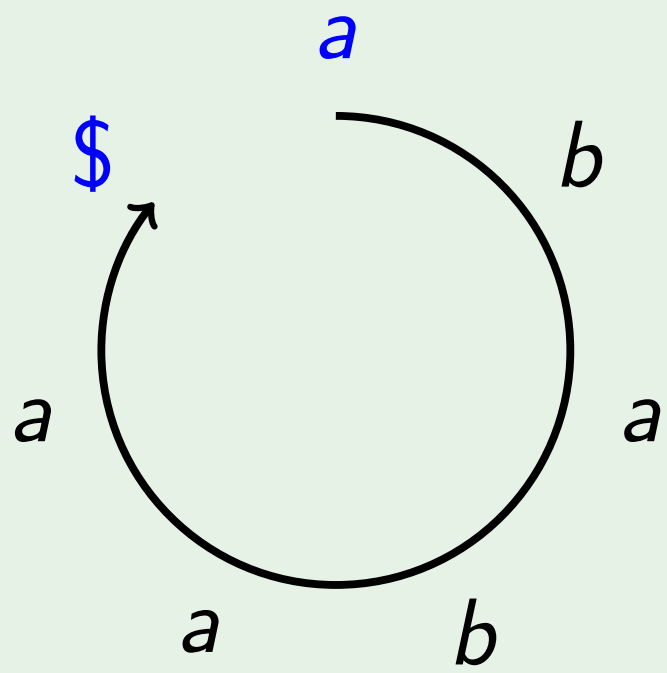
$C_2 = ab$

$C_1 = ba$

$C_3 = ba$

Example

$L = 2$



$C_6 = \$a$

$C_5 = a\$$

$C_4 = aa$

$C_0 = ab$

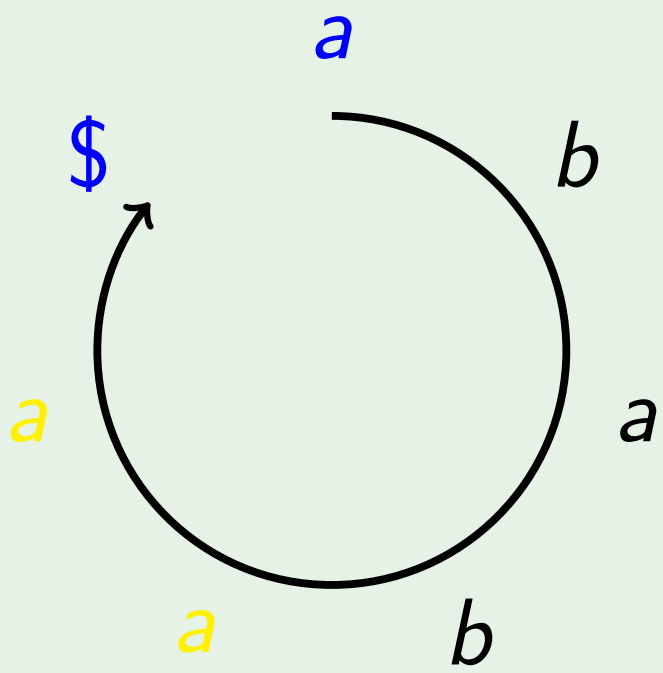
$C_2 = ab$

$C_1 = ba$

$C_3 = ba$

Example

$L = 2$



$C'_4 = aa\$a$

$C_5 = a\$$

$C_4 = aa$

$C_0 = ab$

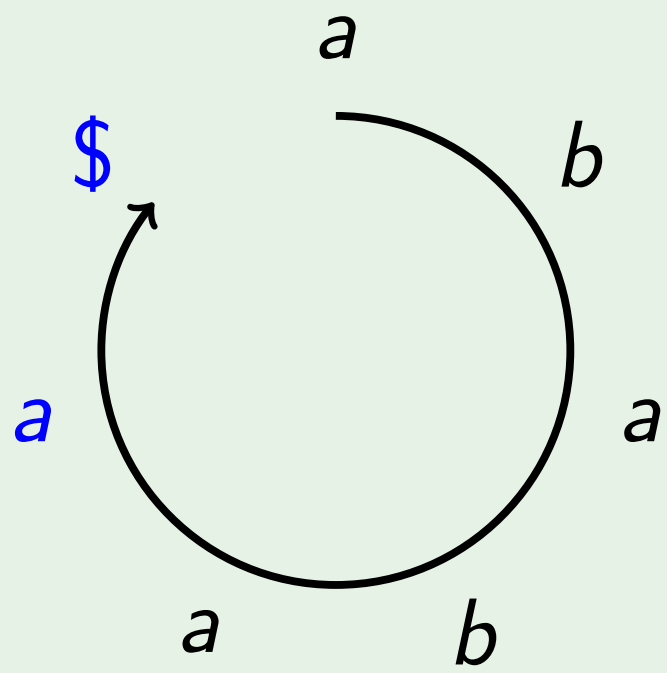
$C_2 = ab$

$C_1 = ba$

$C_3 = ba$

Example

$L = 2$



$C'_4 = aa\$a$

$C_5 = a\$$

$C_4 = aa$

$C_0 = ab$

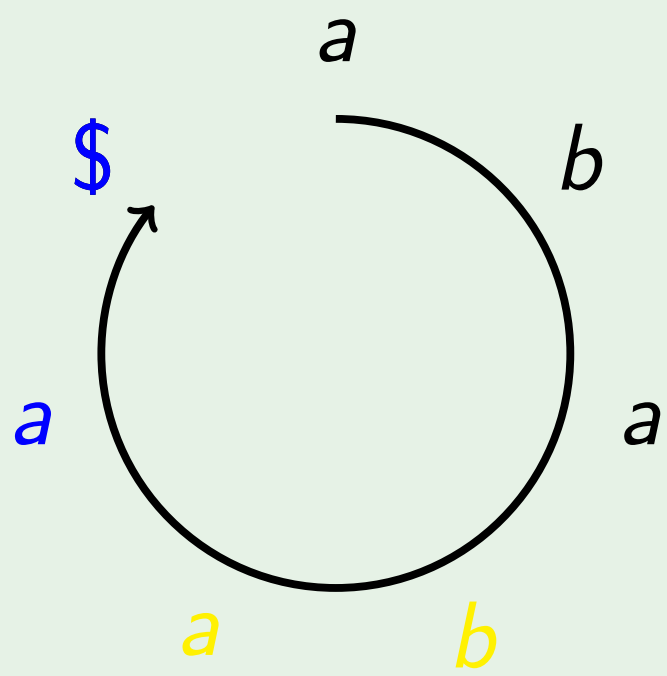
$C_2 = ab$

$C_1 = ba$

$C_3 = ba$

Example

$L = 2$



$C'_4 = aa\$a$

$C'_3 = baa\$$

$C_4 = aa$

$C_0 = ab$

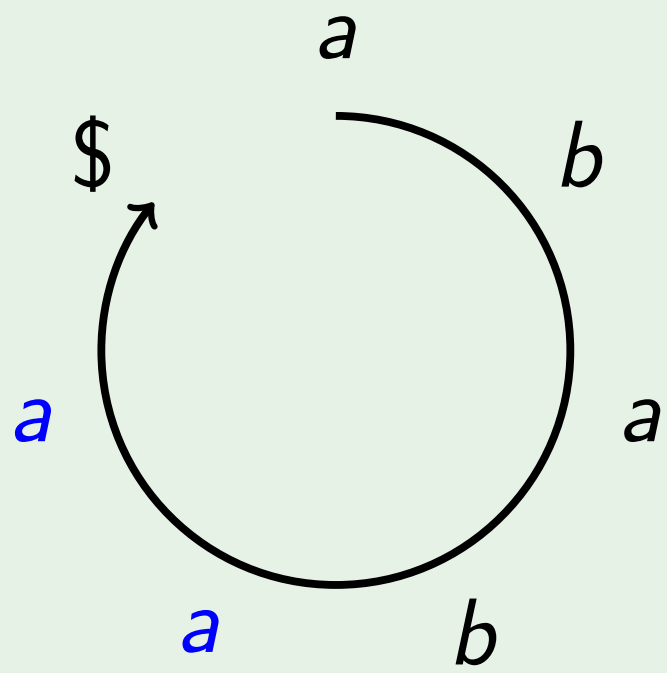
$C_2 = ab$

$C_1 = ba$

$C_3 = ba$

Example

$L = 2$



$C'_4 = aa\$a$

$C'_3 = baa\$$

$C_4 = aa$

$C_0 = ab$

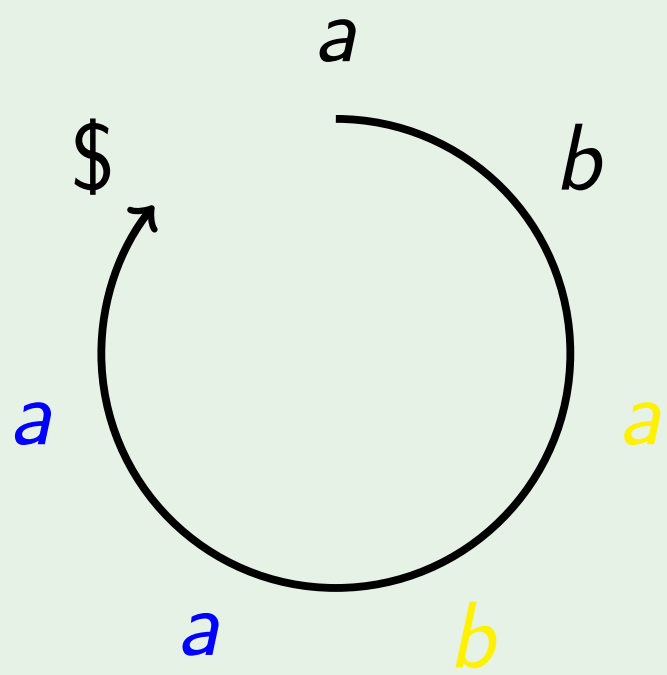
$C_2 = ab$

$C_1 = ba$

$C_3 = ba$

Example

$L = 2$



$C'_4 = aa\$a$

$C'_3 = baa\$$

$C'_2 = abaa$

$C_0 = ab$

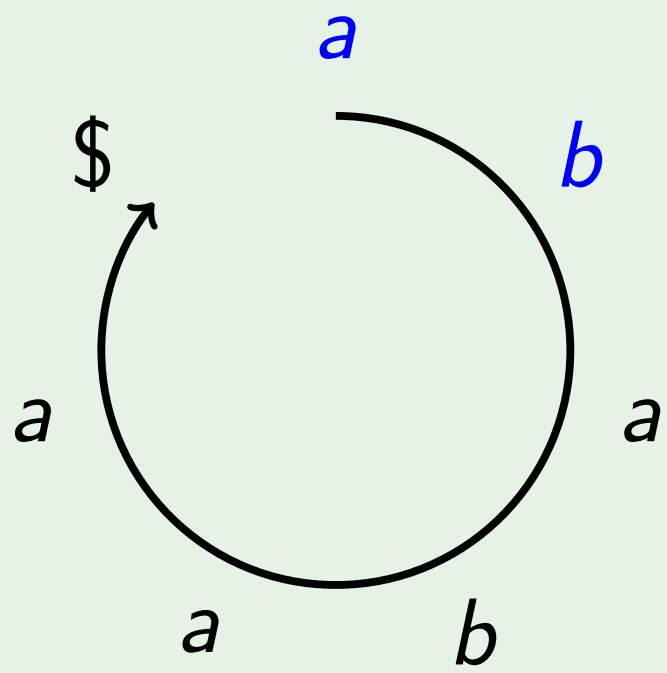
$C_2 = ab$

$C_1 = ba$

$C_3 = ba$

Example

$L = 2$



$C'_4 = aa\$a$

$C'_3 = baa\$$

$C'_2 = abaa$

$C_0 = ab$

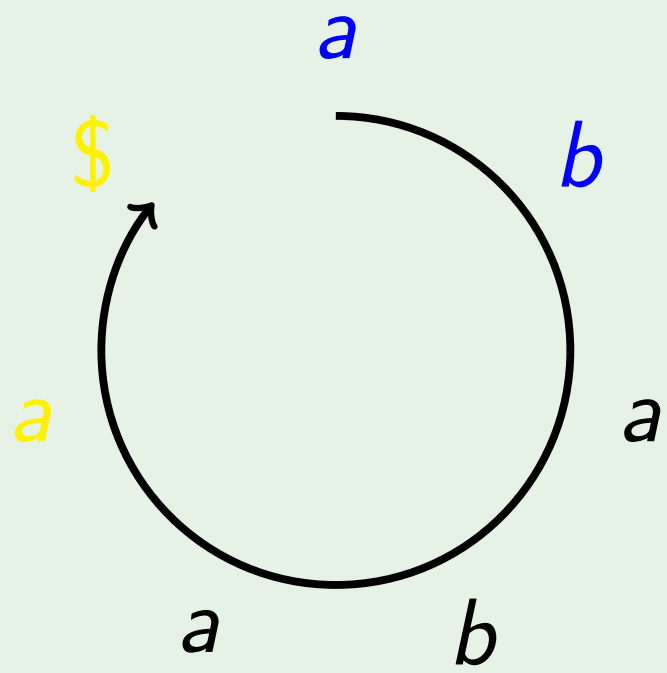
$C_2 = ab$

$C_1 = ba$

$C_3 = ba$

Example

$L = 2$



$C'_4 = aa\$a$

$C'_3 = baa\$$

$C'_2 = abaa$

$C'_5 = a\$ab$

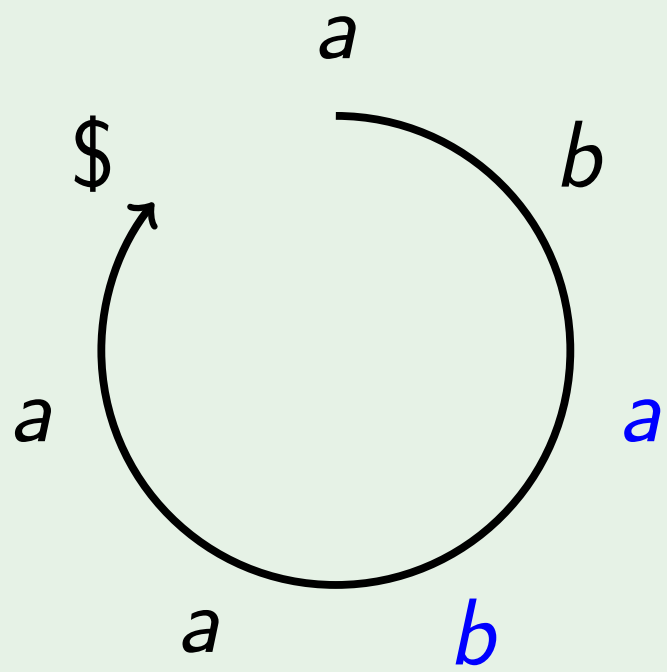
$C_2 = ab$

$C_1 = ba$

$C_3 = ba$

Example

$L = 2$



$C'_4 = aa\$a$

$C'_3 = baa\$$

$C'_2 = abaa$

$C'_5 = a\$ab$

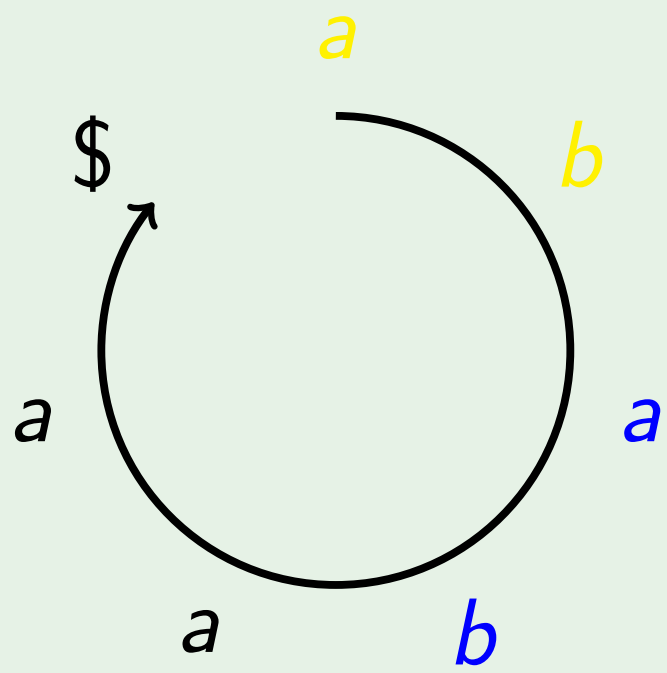
$C_2 = ab$

$C_1 = ba$

$C_3 = ba$

Example

$L = 2$



$C'_4 = aa\$a$

$C'_3 = baa\$$

$C'_2 = abaa$

$C'_5 = a\$ab$

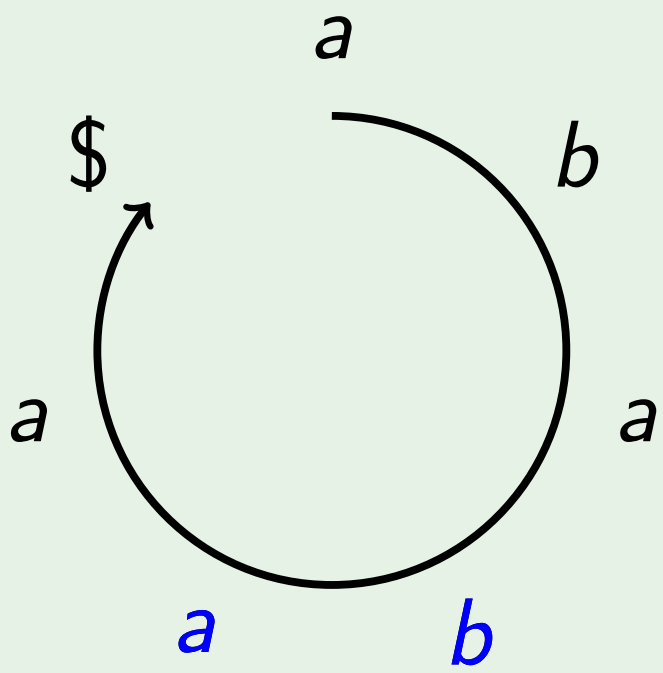
$C'_0 = abab$

$C_1 = ba$

$C_3 = ba$

Example

$L = 2$



$C'_4 = aa\$a$

$C'_3 = baa\$$

$C'_2 = abaa$

$C'_5 = a\$ab$

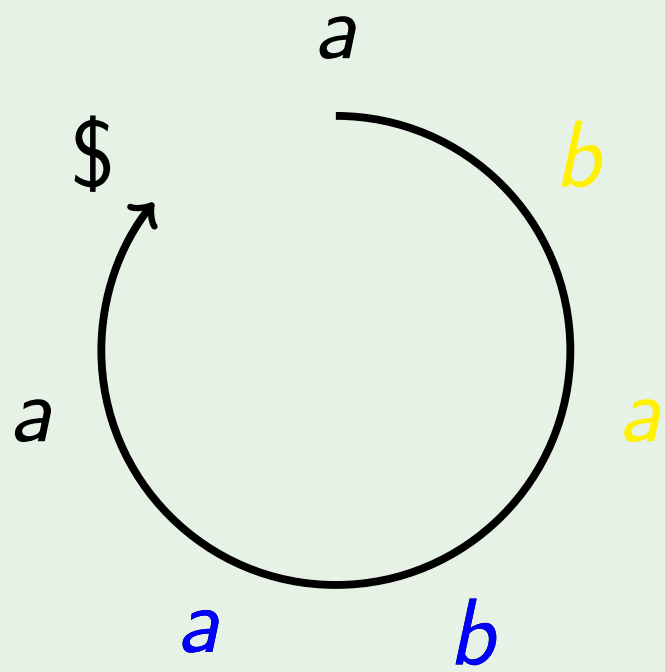
$C'_0 = abab$

$C_1 = ba$

$C_3 = ba$

Example

$L = 2$



$C'_4 = aa\$a$

$C'_3 = baa\$$

$C'_2 = abaa$

$C'_5 = a\$ab$

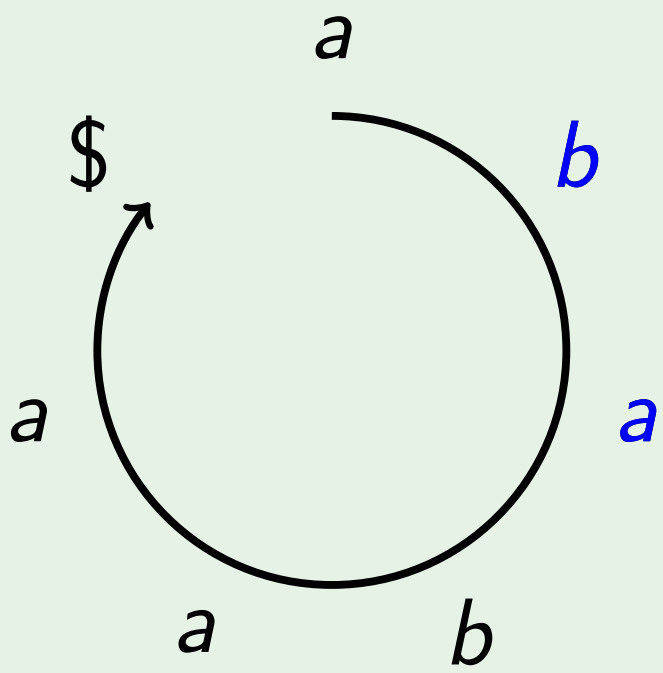
$C'_0 = abab$

$C'_6 = \$aba$

$C_3 = ba$

Example

$L = 2$



$C'_4 = aa\$a$

$C'_3 = baa\$$

$C'_2 = abaa$

$C'_5 = a\$ab$

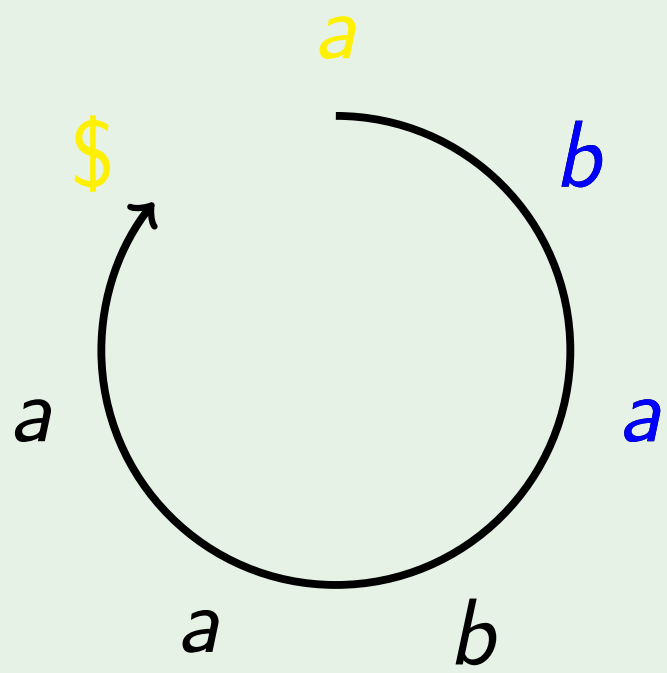
$C'_0 = abab$

$C'_6 = \$aba$

$C_3 = ba$

Example

$L = 2$



$C'_4 = aa\$a$

$C'_3 = baa\$$

$C'_2 = abaa$

$C'_5 = a\$ab$

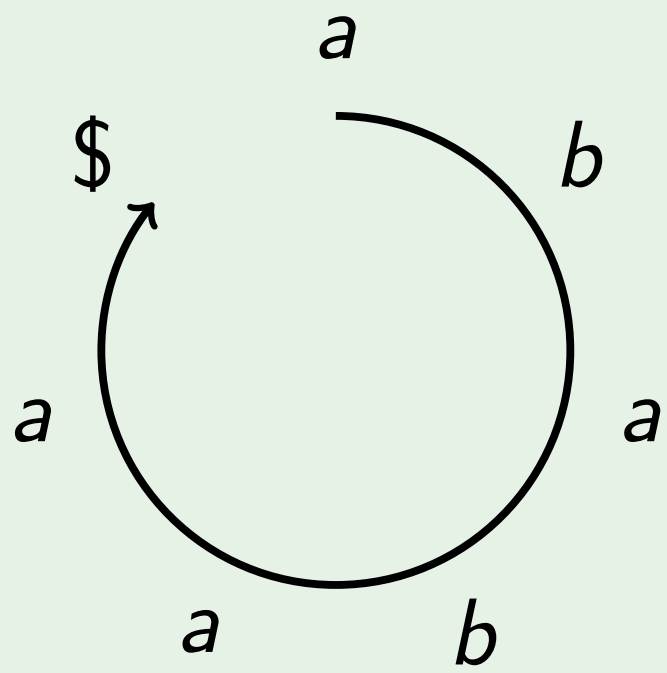
$C'_0 = abab$

$C'_6 = \$aba$

$C'_1 = baba$

Example

$L = 2$



$C'_4 = aa\$a$

$C'_3 = baa\$$

$C'_2 = abaa$

$C'_5 = a\$ab$

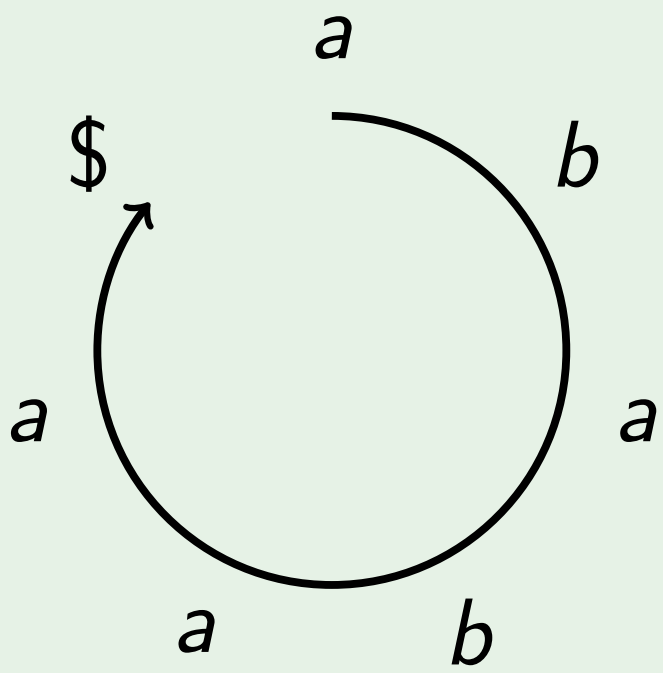
$C'_0 = abab$

$C'_6 = \$aba$

$C'_1 = baba$

Example

$L = 2$



$C'_4 = aa\$a$

$C'_3 = baa\$$

$C'_2 = abaa$

$C'_5 = a\$ab$

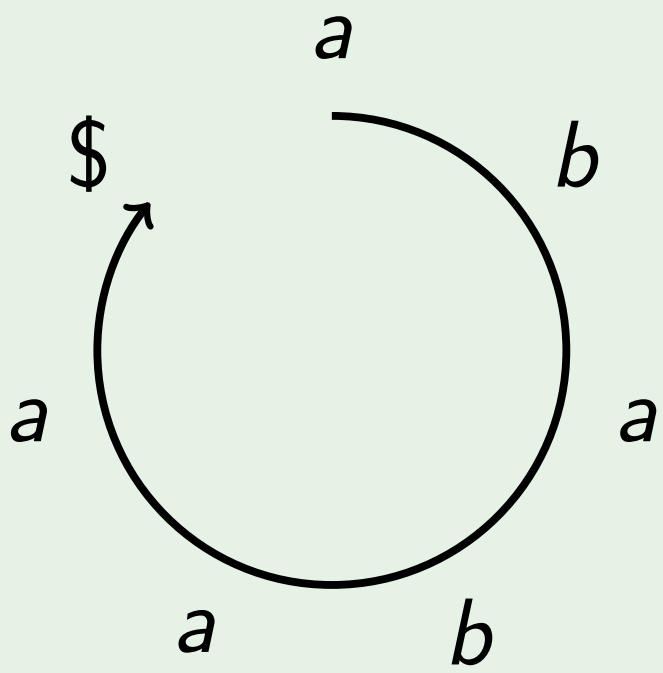
$C'_0 = abab$

$C'_6 = \$aba$

$C'_1 = baba$

Example

$L = 2$



$C'_4 = aa\$a$

$C'_3 = baa\$$

$C'_2 = abaa$

$C'_5 = a\$ab$

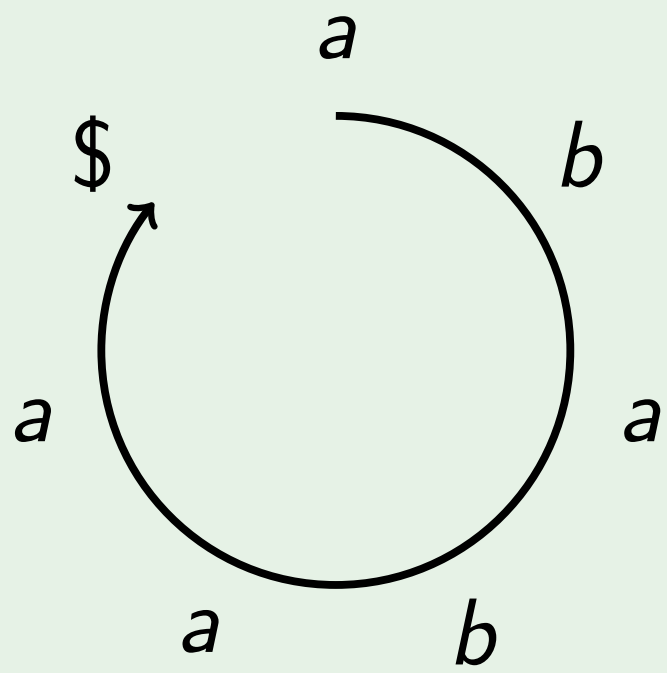
$C'_0 = abab$

$C'_6 = \$aba$

$C'_1 = baba$

Example

$L = 2$



$C'_6 = \$aba$

$C'_5 = a\$ab$

$C'_4 = aa\$a$

$C'_2 = abaa$

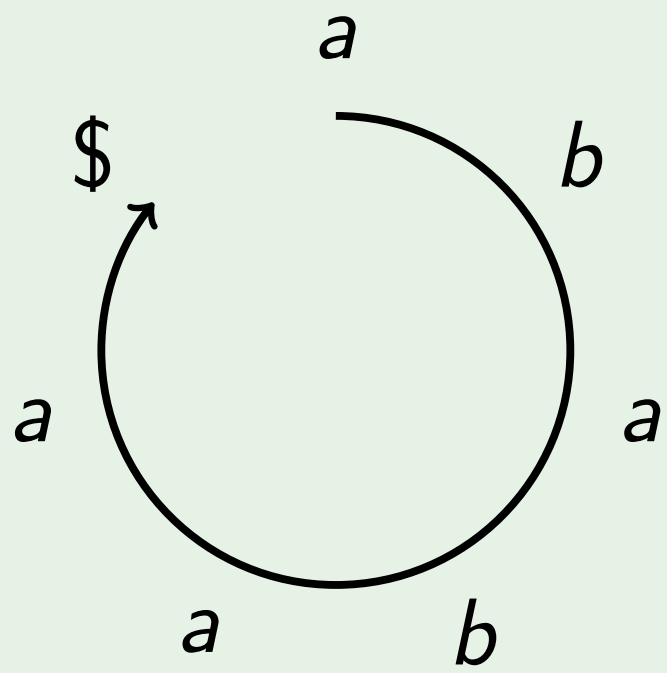
$C'_0 = abab$

$C'_3 = baa\$$

$C'_1 = baba$

Example

$L = 2$



$C'_6 = \$aba$

$C'_5 = a\$ab$

$C'_4 = aa\$a$

$C'_2 = abaa$

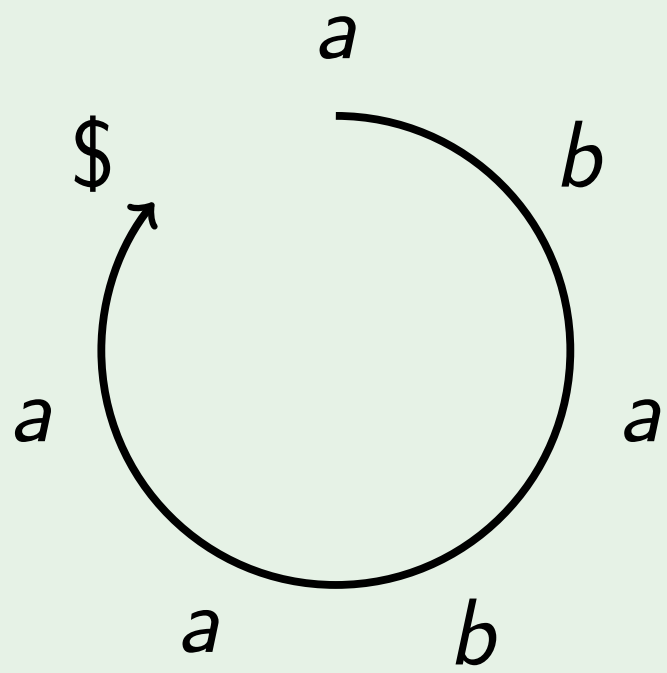
$C'_0 = abab$

$C'_3 = baa\$$

$C'_1 = baba$

Example

$L = 2$



$C'_6 = \$aba$

$C'_5 = a\$ab$

$C'_4 = aa\$a$

$C'_2 = abaa$

$C'_0 = abab$

$C'_3 = baa\$$

$C'_1 = baba$

Sorting doubled cyclic shifts

- C'_i — doubled cyclic shift starting in i

Sorting doubled cyclic shifts

- C'_i — doubled cyclic shift starting in i
- C'_i is a pair (C_i, C_{i+L})

Sorting doubled cyclic shifts

- C'_i — doubled cyclic shift starting in i
- C'_i is a pair (C_i, C_{i+L})
- $C_{order[0]}, C_{order[1]}, \ldots, C_{order[|S|-1]}$ are already sorted

Sorting doubled cyclic shifts

- C'_i — doubled cyclic shift starting in i
- C'_i is a pair (C_i, C_{i+L})
- $C_{order[0]}, C_{order[1]}, \ldots, C_{order[|S|-1]}$ are already sorted
- Take doubled cyclic shifts starting exactly L counter-clockwise ("to the left")

Sorting doubled cyclic shifts

- C'_i — doubled cyclic shift starting in i
- C'_i is a pair (C_i, C_{i+L})
- $C_{order[0]}, C_{order[1]}, \ldots, C_{order[|S|-1]}$ are already sorted
- Take doubled cyclic shifts starting exactly L counter-clockwise ("to the left")
- $C'_{order[0]-L}, C'_{order[1]-L}, \ldots, C_{order[|S|-1]-L}$ are sorted by second element of pair

Sorting doubled cyclic shifts

- $C'_{order[0]-L}, C'_{order[1]-L}, \ldots, C_{order[|S|-1]-L}$ are sorted by second element of pair
- Need a stable sort by first elements of pairs
- Counting sort is stable!
- We know equivalence classes of single shifts for counting sort

Outline

1. Suffix Array
2. General Construction Strategy
3. Initialization
4. Sort Doubled Cyclic Shifts
5. Implement SortDoubled
6. Updating Classes
7. Implement UpdateClasses
8. Building Suffix Array

SortDoubled(S, L, *order*, *class*)

```
count ← zero array of size |S|
newOrder ← array of size |S|
for i from 0 to |S| − 1:
   count[class[i]] ← count[class[i]] + 1
for j from 1 to |S| − 1:
   count[j] ← count[j] + count[j − 1]
for i from |S| − 1 down to 0:
   start ← (order[i] − L + |S|) mod |S|
   cl ← class[start]
   count[cl] ← count[cl] − 1
   newOrder[count[cl]] ← start
return newOrder
```

SortDoubled(S, L, *order*, *class*)

```
count ← zero array of size |S|
newOrder ← array of size |S|
for i from 0 to |S| − 1:
   count[class[i]] ← count[class[i]] + 1
for j from 1 to |S| − 1:
   count[j] ← count[j] + count[j − 1]
for i from |S| − 1 down to 0:
   start ← (order[i] − L + |S|) mod |S|
   cl ← class[start]
   count[cl] ← count[cl] − 1
   newOrder[count[cl]] ← start
return newOrder
```

SortDoubled(S, L, *order*, *class*)

```
count ← zero array of size |S|
newOrder ← array of size |S|
for i from 0 to |S| − 1:
    count[class[i]] ← count[class[i]] + 1
for j from 1 to |S| − 1:
    count[j] ← count[j] + count[j − 1]
for i from |S| − 1 down to 0:
    start ← (order[i] − L + |S|)  mod |S|
    cl ← class[start]
    count[cl] ← count[cl] − 1
    newOrder[count[cl]] ← start
return newOrder
```

SortDoubled(S, L, *order*, *class*)

```
count ← zero array of size |S|
newOrder ← array of size |S|
for i from 0 to |S| − 1:
    count[class[i]] ← count[class[i]] + 1
for j from 1 to |S| − 1:
    count[j] ← count[j] + count[j − 1]
for i from |S| − 1 down to 0:
    start ← (order[i] − L + |S|)  mod |S|
    cl ← class[start]
    count[cl] ← count[cl] − 1
    newOrder[count[cl]] ← start
return newOrder
```

SortDoubled(S, L, *order*, *class*)

```
count ← zero array of size |S|
newOrder ← array of size |S|
for i from 0 to |S| − 1:
    count[class[i]] ← count[class[i]] + 1
for j from 1 to |S| − 1:
    count[j] ← count[j] + count[j − 1]
for i from |S| − 1 down to 0:
    start ← (order[i] − L + |S|) mod |S|
    cl ← class[start]
    count[cl] ← count[cl] − 1
    newOrder[count[cl]] ← start
return newOrder
```

SortDoubled(S, L, *order*, *class*)

```
count ← zero array of size |S|
newOrder ← array of size |S|
for i from 0 to |S| − 1:
   count[class[i]] ← count[class[i]] + 1
for j from 1 to |S| − 1:
   count[j] ← count[j] + count[j − 1]
for i from |S| − 1 down to 0:
   start ← (order[i] − L + |S|)  mod |S|
   cl ← class[start]
   count[cl] ← count[cl] − 1
   newOrder[count[cl]] ← start
return newOrder
```

SortDoubled(S, L, *order*, *class*)

```
count ← zero array of size |S|
newOrder ← array of size |S|
for i from 0 to |S| − 1:
   count[class[i]] ← count[class[i]] + 1
for j from 1 to |S| − 1:
   count[j] ← count[j] + count[j − 1]
for i from |S| − 1 down to 0:
   start ← (order[i] − L + |S|) mod |S|
   cl ← class[start]
   count[cl] ← count[cl] − 1
   newOrder[count[cl]] ← start
return newOrder
```

SortDoubled(S, L, *order*, *class*)

```
count ← zero array of size |S|
newOrder ← array of size |S|
for i from 0 to |S| − 1:
   count[class[i]] ← count[class[i]] + 1
for j from 1 to |S| − 1:
   count[j] ← count[j] + count[j − 1]
for i from |S| − 1 down to 0:
   start ← (order[i] − L + |S|) mod |S|
   cl ← class[start]
   count[cl] ← count[cl] − 1
   newOrder[count[cl]] ← start
return newOrder
```

SortDoubled(S, L, *order*, *class*)

```
count ← zero array of size |S|
newOrder ← array of size |S|
for i from 0 to |S| − 1:
   count[class[i]] ← count[class[i]] + 1
for j from 1 to |S| − 1:
   count[j] ← count[j] + count[j − 1]
for i from |S| − 1 down to 0:
   start ← (order[i] − L + |S|) mod |S|
   cl ← class[start]
   count[cl] ← count[cl] − 1
   newOrder[count[cl]] ← start
return newOrder
```

SortDoubled(S, L, *order*, *class*)

```
count ← zero array of size |S|
newOrder ← array of size |S|
for i from 0 to |S| − 1:
   count[class[i]] ← count[class[i]] + 1
for j from 1 to |S| − 1:
   count[j] ← count[j] + count[j − 1]
for i from |S| − 1 down to 0:
   start ← (order[i] − L + |S|) mod |S|
   cl ← class[start]
   count[cl] ← count[cl] − 1
   newOrder[count[cl]] ← start
return newOrder
```

Lemma

The running time of `SortDoubled` is $O(|S|)$.

Proof

Three for loops with $O(|S|)$ iterations each. □

Outline

1. Suffix Array
2. General Construction Strategy
3. Initialization
4. Sort Doubled Cyclic Shifts
5. Implement SortDoubled
6. Updating Classes
7. Implement UpdateClasses
8. Building Suffix Array

Updating classes

- Pairs are sorted — go through them in order, if a pair is different from previous, put it into a new class, otherwise put it into previous class
- $$(P_1, P_2) == (Q_1, Q_2) \Leftrightarrow$$
$$(P_1 == Q_1) \text{ and } (P_2 == Q_2)$$
- We know equivalence classes of elements of pairs

Example

$S = ababaa\$$

C'_6 $\$a$ $(0,1)$ $\leftarrow$ $class = [1,2,1,2,1,1,0]$

C'_5 $a\$$ $(1,0)$ $newOrder = [6,5,4,0,2,1,3]$

C'_4 aa $(1,1)$ $newClass = [\ ,\ ,\ ,\ ,\ ,\ ,\ ]$

C'_0 ab $(1,2)$

C'_2 ab $(1,2)$

C'_1 ba $(2,1)$

C'_3 ba $(2,1)$

Example

$S = ababaa\$$

C'_6 $\$a$ $(0,1)$ $\leftarrow$ $class = [1,2,1,2,1,1,0]$

C'_5 $a\$$ $(1,0)$ $newOrder = [6,5,4,0,2,1,3]$

C'_4 aa $(1,1)$ $newClass = [\ ,\ ,\ ,\ ,\ ,\ ,\ ]$

C'_0 ab $(1,2)$

C'_2 ab $(1,2)$

C'_1 ba $(2,1)$

C'_3 ba $(2,1)$

Example

$S = ababaa\$$

C'_6 $\$a$ $(0,1)$ $\leftarrow$ $class = [1,2,1,2,1,1,0]$

C'_5 $a\$$ $(1,0)$ $newOrder = [6,5,4,0,2,1,3]$

C'_4 aa $(1,1)$ $newClass = [\ ,\ ,\ ,\ ,\ ,\ ,0]$

C'_0 ab $(1,2)$

C'_2 ab $(1,2)$

C'_1 ba $(2,1)$

C'_3 ba $(2,1)$

Example

$S = ababaa\$$

C'_6 $\$a$ $(0,1)$ $\leftarrow$ $class = [1,2,1,2,1,1,0]$

C'_5 $a\$$ $(1,0)$ $newOrder = [6,5,4,0,2,1,3]$

C'_4 aa $(1,1)$ $newClass = [\ ,\ ,\ ,\ ,\ ,\ ,0]$

C'_0 ab $(1,2)$

C'_2 ab $(1,2)$

C'_1 ba $(2,1)$

C'_3 ba $(2,1)$

Example

$S = ababaa\$$

C'_6 $\$a$ $(0,1)$ $\leftarrow$ $class = [1,2,1,2,1,1,0]$

C'_5 $a\$$ $(1,0)$ $newOrder = [6,5,4,0,2,1,3]$

C'_4 aa $(1,1)$ $newClass = [\ ,\ ,\ ,\ ,\ ,1,0]$

C'_0 ab $(1,2)$

C'_2 ab $(1,2)$

C'_1 ba $(2,1)$

C'_3 ba $(2,1)$

Example

$S = ababaa\$$

C'_6 $\$a$ $(0,1)$ $\leftarrow$ $class = [1,2,1,2,1,1,0]$

C'_5 $a\$$ $(1,0)$ $newOrder = [6,5,4,0,2,1,3]$

C'_4 aa $(1,1)$ $newClass = [\ ,\ ,\ ,\ ,\ ,1,0]$

C'_0 ab $(1,2)$

C'_2 ab $(1,2)$

C'_1 ba $(2,1)$

C'_3 ba $(2,1)$

Example

$S = ababaa\$$

C'_6 $\$a$ $(0,1) \leftarrow class = [1,2,1,2,1,1,0]$

C'_5 $a\$$ $(1,0)$ $newOrder = [6,5,4,0,2,1,3]$

C'_4 aa $(1,1)$ $newClass = [\ ,\ ,\ ,\ ,2,1,0]$

C'_0 ab $(1,2)$

C'_2 ab $(1,2)$

C'_1 ba $(2,1)$

C'_3 ba $(2,1)$

Example

$S = ababaa\$$

C'_6 $\$a$ $(0,1)$ $\leftarrow$ $class = [1,2,1,2,1,1,0]$

C'_5 $a\$$ $(1,0)$ $newOrder = [6,5,4,0,2,1,3]$

C'_4 aa $(1,1)$ $newClass = [\ ,\ ,\ ,\ ,2,1,0]$

C'_0 ab $(1,2)$

C'_2 ab $(1,2)$

C'_1 ba $(2,1)$

C'_3 ba $(2,1)$

Example

$S = ababaa\$$

C'_6 $\$a$ $(0,1) \leftarrow class = [1,2,1,2,1,1,0]$

C'_5 $a\$$ $(1,0)$ $newOrder = [6,5,4,0,2,1,3]$

C'_4 aa $(1,1)$ $newClass = [3, \ , \ , \ , 2,1,0]$

C'_0 ab $(1,2)$

C'_2 ab $(1,2)$

C'_1 ba $(2,1)$

C'_3 ba $(2,1)$

Example

$S = ababaa\$$

C'_6 $\$a$ $(0,1) \leftarrow class = [1,2,1,2,1,1,0]$

C'_5 $a\$$ $(1,0)$ $newOrder = [6,5,4,0,2,1,3]$

C'_4 aa $(1,1)$ $newClass = [3, \ , \ , \ , 2,1,0]$

C'_0 ab $(1,2)$

C'_2 ab $(1,2)$

C'_1 ba $(2,1)$

C'_3 ba $(2,1)$

Example

$S = ababaa\$$

C'_6 $\$a$ $(0,1)$ $\leftarrow$ $class = [1,2,1,2,1,1,0]$

C'_5 $a\$$ $(1,0)$ $newOrder = [6,5,4,0,2,1,3]$

C'_4 aa $(1,1)$ $newClass = [3,\ \ ,3,\ \ ,2,1,0]$

C'_0 ab $(1,2)$

C'_2 ab $(1,2)$

C'_1 ba $(2,1)$

C'_3 ba $(2,1)$

Example

$S = ababaa\$$

C'_6 $\$a$ $(0,1)$ $\leftarrow$ $class = [1,2,1,2,1,1,0]$

C'_5 $a\$$ $(1,0)$ $newOrder = [6,5,4,0,2,1,3]$

C'_4 aa $(1,1)$ $newClass = [3,\ \ ,3,\ \ ,2,1,0]$

C'_0 ab $(1,2)$

C'_2 ab $(1,2)$

C'_1 ba $(2,1)$

C'_3 ba $(2,1)$

Example

$S = ababaa\$$

C'_6 $\$a$ $(0,1) \leftarrow class = [1,2,1,2,1,1,0]$

C'_5 $a\$$ $(1,0)$ $newOrder = [6,5,4,0,2,1,3]$

C'_4 aa $(1,1)$ $newClass = [3,4,3,\ \ ,2,1,0]$

C'_0 ab $(1,2)$

C'_2 ab $(1,2)$

C'_1 ba $(2,1)$

C'_3 ba $(2,1)$

Example

$S = ababaa\$$

C'_6 $\$a$ $(0,1) \leftarrow class = [1,2,1,2,1,1,0]$

C'_5 $a\$$ $(1,0)$ $newOrder = [6,5,4,0,2,1,3]$

C'_4 aa $(1,1)$ $newClass = [3,4,3,\ \ ,2,1,0]$

C'_0 ab $(1,2)$

C'_2 ab $(1,2)$

C'_1 ba $(2,1)$

C'_3 ba $(2,1)$

Example

$S = ababaa\$$

C'_6 $\$a$ $(0,1) \leftarrow class = [1,2,1,2,1,1,0]$

C'_5 $a\$$ $(1,0)$ $newOrder = [6,5,4,0,2,1,3]$

C'_4 aa $(1,1)$ $newClass = [3,4,3,4,2,1,0]$

C'_0 ab $(1,2)$

C'_2 ab $(1,2)$

C'_1 ba $(2,1)$

C'_3 ba $(2,1)$

Example

$S = ababaa\$$

C'_6 $\$a$ $(0,1) \leftarrow class = [1,2,1,2,1,1,0]$

C'_5 $a\$$ $(1,0)$ $newOrder = [6,5,4,0,2,1,3]$

C'_4 aa $(1,1)$ $newClass = [3,4,3,4,2,1,0]$

C'_0 ab $(1,2)$

C'_2 ab $(1,2)$

C'_1 ba $(2,1)$

C'_3 ba $(2,1)$

Outline

1. Suffix Array
2. General Construction Strategy
3. Initialization
4. Sort Doubled Cyclic Shifts
5. Implement SortDoubled
6. Updating Classes
7. Implement UpdateClasses
8. Building Suffix Array

UpdateClasses(*newOrder*, *class*, *L*)

```
n ← |newOrder|
newClass ← array of size n
newClass[newOrder[0]] ← 0
for i from 1 to n − 1:
   cur ← newOrder[i], prev ← newOrder[i − 1]
   mid ← (cur + L), midPrev ← (prev + L) (mod n)
   if class[cur] ≠ class[prev] or
         class[mid] ≠ class[midPrev]:
      newClass[cur] ← newClass[prev] + 1
   else:
      newClass[cur] ← newClass[prev]
return newClass
```

UpdateClasses(*newOrder*, *class*, *L*)

```
n ← |newOrder|
newClass ← array of size n
newClass[newOrder[0]] ← 0
for i from 1 to n − 1:
   cur ← newOrder[i], prev ← newOrder[i − 1]
   mid ← (cur + L), midPrev ← (prev + L) (mod n)
   if class[cur] ≠ class[prev] or
         class[mid] ≠ class[midPrev]:
      newClass[cur] ← newClass[prev] + 1
   else:
      newClass[cur] ← newClass[prev]
return newClass
```

UpdateClasses(*newOrder*, *class*, *L*)

```
n ← |newOrder|
newClass ← array of size n
newClass[newOrder[0]] ← 0
for i from 1 to n − 1:
   cur ← newOrder[i], prev ← newOrder[i − 1]
   mid ← (cur + L), midPrev ← (prev + L) (mod n)
   if class[cur] ≠ class[prev] or
         class[mid] ≠ class[midPrev]:
      newClass[cur] ← newClass[prev] + 1
   else:
      newClass[cur] ← newClass[prev]
return newClass
```

UpdateClasses(*newOrder*, *class*, *L*)

```
n ← |newOrder|
newClass ← array of size n
newClass[newOrder[0]] ← 0
for i from 1 to n − 1:
    cur ← newOrder[i], prev ← newOrder[i − 1]
    mid ← (cur + L), midPrev ← (prev + L) (mod n)
    if class[cur] ≠ class[prev] or
            class[mid] ≠ class[midPrev]:
        newClass[cur] ← newClass[prev] + 1
    else:
        newClass[cur] ← newClass[prev]
return newClass
```

UpdateClasses(*newOrder*, *class*, *L*)

```
n ← |newOrder|
newClass ← array of size n
newClass[newOrder[0]] ← 0
for i from 1 to n − 1:
   cur ← newOrder[i], prev ← newOrder[i − 1]
   mid ← (cur + L), midPrev ← (prev + L) (mod n)
   if class[cur] ≠ class[prev] or
         class[mid] ≠ class[midPrev]:
      newClass[cur] ← newClass[prev] + 1
   else:
      newClass[cur] ← newClass[prev]
return newClass
```

UpdateClasses(*newOrder*, *class*, *L*)

```
n ← |newOrder|
newClass ← array of size n
newClass[newOrder[0]] ← 0
for i from 1 to n − 1:
   cur ← newOrder[i], prev ← newOrder[i − 1]
   mid ← (cur + L), midPrev ← (prev + L) (mod n)
   if class[cur] ≠ class[prev] or
         class[mid] ≠ class[midPrev]:
      newClass[cur] ← newClass[prev] + 1
   else:
      newClass[cur] ← newClass[prev]
return newClass
```

UpdateClasses(*newOrder*, *class*, *L*)

```
n ← |newOrder|
newClass ← array of size n
newClass[newOrder[0]] ← 0
for i from 1 to n − 1:
   cur ← newOrder[i], prev ← newOrder[i − 1]
   mid ← (cur + L), midPrev ← (prev + L) (mod n)
   if class[cur] ≠ class[prev] or
         class[mid] ≠ class[midPrev]:
      newClass[cur] ← newClass[prev] + 1
   else:
      newClass[cur] ← newClass[prev]
return newClass
```

UpdateClasses(*newOrder*, *class*, *L*)

```
n ← |newOrder|
newClass ← array of size n
newClass[newOrder[0]] ← 0
for i from 1 to n − 1:
   cur ← newOrder[i], prev ← newOrder[i − 1]
   mid ← (cur + L), midPrev ← (prev + L) (mod n)
   if class[cur] ≠ class[prev] or
         class[mid] ≠ class[midPrev]:
      newClass[cur] ← newClass[prev] + 1
   else:
      newClass[cur] ← newClass[prev]
return newClass
```

UpdateClasses(*newOrder*, *class*, *L*)

```
n ← |newOrder|
newClass ← array of size n
newClass[newOrder[0]] ← 0
for i from 1 to n − 1:
   cur ← newOrder[i], prev ← newOrder[i − 1]
   mid ← (cur + L), midPrev ← (prev + L) (mod n)
   if class[cur] ≠ class[prev] or
         class[mid] ≠ class[midPrev]:
      newClass[cur] ← newClass[prev] + 1
   else:
      newClass[cur] ← newClass[prev]
return newClass
```

UpdateClasses(*newOrder*, *class*, *L*)

```
n ← |newOrder|
newClass ← array of size n
newClass[newOrder[0]] ← 0
for i from 1 to n − 1:
   cur ← newOrder[i], prev ← newOrder[i − 1]
   mid ← (cur + L), midPrev ← (prev + L) (mod n)
   if class[cur] ≠ class[prev] or
         class[mid] ≠ class[midPrev]:
      newClass[cur] ← newClass[prev] + 1
   else:
      newClass[cur] ← newClass[prev]
return newClass
```

UpdateClasses(*newOrder*, *class*, *L*)

```
n ← |newOrder|
newClass ← array of size n
newClass[newOrder[0]] ← 0
for i from 1 to n − 1:
   cur ← newOrder[i], prev ← newOrder[i − 1]
   mid ← (cur + L), midPrev ← (prev + L) (mod n)
   if class[cur] ≠ class[prev] or
         class[mid] ≠ class[midPrev]:
      newClass[cur] ← newClass[prev] + 1
   else:
      newClass[cur] ← newClass[prev]
return newClass
```

Lemma

The running time of `UpdateClasses` is $O(|S|)$.

Proof

One for loop with $O(|S|)$ iterations. □

Outline

1. Suffix Array
2. General Construction Strategy
3. Initialization
4. Sort Doubled Cyclic Shifts
5. Implement SortDoubled
6. Updating Classes
7. Implement UpdateClasses
8. Building Suffix Array

BuildSuffixArray(S)

```
order ← SortCharacters(S)
class ← ComputeCharClasses(S, order)
L ← 1
while L < |S|:
    order ← SortDoubled(S, L, order, class)
    class ← UpdateClasses(order, class, L)
    L ← 2L
return order
```

BuildSuffixArray(S)

```
order ← SortCharacters(S)
class ← ComputeCharClasses(S, order)
L ← 1
while L < |S|:
    order ← SortDoubled(S, L, order, class)
    class ← UpdateClasses(order, class, L)
    L ← 2L
return order
```

BuildSuffixArray(S)

```
order ← SortCharacters(S)
class ← ComputeCharClasses(S, order)
L ← 1
while L < |S|:
    order ← SortDoubled(S, L, order, class)
    class ← UpdateClasses(order, class, L)
    L ← 2L
return order
```

BuildSuffixArray(S)

```
order ← SortCharacters(S)
class ← ComputeCharClasses(S, order)
L ← 1
while L < |S|:
    order ← SortDoubled(S, L, order, class)
    class ← UpdateClasses(order, class, L)
    L ← 2L
return order
```

BuildSuffixArray(S)

```
order ← SortCharacters(S)
class ← ComputeCharClasses(S, order)
L ← 1
while L < |S|:
    order ← SortDoubled(S, L, order, class)
    class ← UpdateClasses(order, class, L)
    L ← 2L
return order
```

BuildSuffixArray(S)

```
order ← SortCharacters(S)
class ← ComputeCharClasses(S, order)
L ← 1
while L < |S|:
    order ← SortDoubled(S, L, order, class)
    class ← UpdateClasses(order, class, L)
    L ← 2L
return order
```

BuildSuffixArray(S)

```
order ← SortCharacters(S)
class ← ComputeCharClasses(S, order)
L ← 1
while L < |S|:
    order ← SortDoubled(S, L, order, class)
    class ← UpdateClasses(order, class, L)
    L ← 2L
return order
```

BuildSuffixArray(S)

$order \leftarrow$ SortCharacters(S)
$class \leftarrow$ ComputeCharClasses(S, $order$)
$L \leftarrow 1$
while $L < |S|$:
    $order \leftarrow$ SortDoubled(S, L, $order$, $class$)
    $class \leftarrow$ UpdateClasses($order$, $class$, L)
    $L \leftarrow 2L$
return $order$

BuildSuffixArray(S)

```
order ← SortCharacters(S)
class ← ComputeCharClasses(S, order)
L ← 1
while L < |S|:
    order ← SortDoubled(S, L, order, class)
    class ← UpdateClasses(order, class, L)
    L ← 2L
return order
```

Lemma

The running time of `BuildSuffixArray` is $O(|S| \log |S| + |\Sigma|)$.

Proof

- Initialization: `SortCharacters` in $O(|S| + |\Sigma|)$ and `ComputeCharClasses` in $O(|S|)$

Lemma

The running time of `BuildSuffixArray` is $O(|S| \log |S| + |\Sigma|)$.

Proof

- Initialization: `SortCharacters` in $O(|S| + |\Sigma|)$ and `ComputeCharClasses` in $O(|S|)$
- While loop iteration: `SortDoubled` and `UpdateClasses` run in $O(|S|)$

Lemma

The running time of `BuildSuffixArray` is $O(|S| \log |S| + |\Sigma|)$.

Proof

- Initialization: `SortCharacters` in $O(|S| + |\Sigma|)$ and `ComputeCharClasses` in $O(|S|)$
- While loop iteration: `SortDoubled` and `UpdateClasses` run in $O(|S|)$
- $O(\log |S|)$ iterations while $L < |S|$ □

Conclusion

- Can build suffix array of a string S in $O(|S| \log |S|)$ using $O(|S|)$ memory
- Can also sort all cyclic shifts of a string S in $O(|S| \log |S|)$
- Suffix array enables many fast operations with the string
- Next lesson you will learn to construct suffix tree from suffix array in $O(|S|)$ time, so you will be able to build suffix tree in total $O(|S| \log |S|)$ time!